

Life Planner

Architecture & Integration Roadmap

PWA, VPS Deployment, and a Pluggable Data-Integration Hub

Prepared 2026-06-16

1 System Overview: Three Services

The application runs as three cooperating services. Each layer depends on the one below it.

The three running services

1. **PostgreSQL** (port 5432) — the database and source of truth.
2. **Backend** — FastAPI via `uvicorn` (port 8000); reads and writes Postgres.
3. **Frontend** — Vite dev server (port 5173); the browser UI that calls the backend.

The `start.sh` script orchestrates all three for local development: it ensures Postgres is up, runs database migrations, launches the backend (waiting for port 8000), launches the frontend (waiting for port 5173), then monitors both and shuts everything down cleanly on `Ctrl+C`.

Fixed bug: `start.sh` previously ran `alembic upgrade head -q`. The `-q` flag is not valid for Alembic, so the migration step errored and the script aborted before starting any servers. Removing `-q` (matching `setup.sh`) restored it.

2 Progressive Web App (PWA)

A PWA is the existing frontend plus two pieces:

- **Web App Manifest** — name, icons, theme color, `display: standalone`; this makes the app *installable* (Add to Home Screen, fullscreen launch).
- **Service Worker** — a background script that intercepts network requests and serves cached assets; this enables *offline* use and faster loads.

With Vite these are generated by the `vite-plugin-pwa` plugin (a Workbox wrapper) rather than written by hand.

2.1 The key caveat: a PWA caches the frontend, not the data

Layer	Lives where	Cacheable?
App shell (HTML/JS/CSS, icons)	Static build output	Yes
Planner data (tasks, focuses, areas)	Postgres, via <code>/api</code> on <code>:8000</code>	Partial

With a basic setup the app loads instantly and offline (the shell is cached), but any `/api` call still needs the backend and Postgres. True offline support means caching API responses in IndexedDB, reading from it when offline, queuing writes, and syncing on reconnect — a separate, larger project.

- **Tier 1 (small):** installable app + cached shell + cached GET responses. Feels native; still needs the backend for fresh data and writes.
- **Tier 2 (large):** full local-first / offline-write sync via IndexedDB.

Decision: start with **no offline support** — Tier 1 only.

3 VPS Deployment

Deploying to a VPS resolves the PWA plumbing problems.

The VPS solves	The VPS does <i>not</i> solve
HTTPS via a domain + Let's Encrypt, so service workers register and the PWA is installable anywhere	Offline data (Tier 2) — identical problem; a remote backend makes caching <i>more</i> valuable but no easier
Single origin: a reverse proxy serves <code>dist/</code> and proxies <code>/api</code> on one domain	
Public reachability over mobile data, not just home Wi-Fi	

3.1 Production runtime differs from `start.sh`

`start.sh` is a development runner (`-reload`, Vite dev server, crash-monitor loop). Production on a VPS looks like:

- **Frontend:** `vite build` → static `dist/`, served by Caddy/nginx.
- **Backend:** `uvicorn/gunicorn` under `systemd` (auto-restart), no `-reload`.
- **Postgres:** already a `systemd` service.
- **Reverse proxy:** Caddy terminates HTTPS, serves static, proxies `/api`.

4 Mobile, Health & AI Integration

The governing insight

The backend (FastAPI + Postgres) stays as the **hub and source of truth**. Samsung Health and the OS assistant are **on-device APIs, not cloud APIs** — there is no server-to-server health API the VPS can call. A device-native layer reads the data and pushes it to the backend over HTTP:

[native: Health Connect / assistant] → HTTP → [FastAPI on VPS] → [Postgres]

4.1 Where each integration lands

Goal	From a PWA?	What it really requires
Installable app, push notifications	Yes	PWA on the VPS (HTTPS)
Read Samsung Health steps/activity	No	Native Android reading Health Connect
Smartwatch (Galaxy Watch)	No	Native Wear OS app (Kotlin/Compose)
“Gemini aware of it & manages it”	Partial	Mostly <i>your own</i> in-app AI, not system Gemini

4.2 Samsung Health / activity goals

Modern Android health data flows through **Health Connect** (Google’s on-device store that Samsung Health writes into).

- It is a **device API**, reachable only from a native Android app (or a cross-platform wrapper with a plugin) — not from a web PWA.
- Production read access needs a **Google review / allowlist** (privacy policy, declared permissions).
- Flow: native app reads steps/workouts → POSTs to a backend endpoint → backend matches them to activity-type goals and updates progress.

4.3 Gemini / “aware and manages it”

- **System Gemini managing a third-party app:** largely not openly available. Limited surface via *App Actions / App Functions* (the assistant can *launch* app flows via intents); deep control is gated.
- **Your own in-app assistant** (fully in your control): expose the FastAPI endpoints as **tools / function-calling** to an LLM (the Claude API with tool use), letting users chat to create and manage goals. The clean backend API is what makes this straightforward.

4.4 Mobile app + smartwatch

- A PWA installs on the phone but cannot reach Health Connect, sensors, or Wear OS. Galaxy Watch (4+) runs Wear OS; watch apps are native — there is no “PWA on a watch.”
- Lowest-effort path: **wrap the existing React frontend in Capacitor** → native Android shell + a Health Connect plugin, reusing the current UI. A small Wear OS companion syncs simple data via the backend.

5 Data Integrations: A Pluggable Catalog

The long-term value of this project is concentration: many personal-data sources, each feeding goals, combined in one place. The differentiator is not any single integration but the **hub** — and a clean pattern for adding more over time.

Two integration patterns

Every external source falls into one of two categories, and which one decides how hard it is to build:

1. **Server-side (cloud API).** The VPS calls the provider directly over HTTP. No phone involved. Often only a *username* (public data) or a one-time OAuth token. *Easiest to ship.* Example: **GitHub**.
2. **Device-side (on-device API).** Only a native app on the phone can read it; it then pushes to the backend. Needs the Capacitor/native layer and OS permissions. Example: **Health Connect, screen time**.

5.1 GitHub — a server-side example (easy win)

GitHub exposes a full REST/GraphQL API, so the backend can pull contribution data on a schedule with **no device involved**.

- **Public data** (contribution graph, public commits, public repos) needs only a **username** — ideal for a zero-friction “coding streak” goal.
- **Private activity** (private commits, full counts) needs a personal access token or OAuth.
- **Flow:** a scheduled backend job hits the GitHub API → updates “commit / project” goals (e.g. *commit every day, N PRs this month*).

5.2 Screen time — device-side, with a server-side shortcut

Screen-time goals (e.g. *under 5 hours/day*) work as described: set the target on the website, and a native layer reads usage and reports back.

- **Android (device-side):** `UsageStatsManager` / Digital Wellbeing, gated by the special `PACKAGE_USAGE_STATS` permission the user grants in Settings. The Capacitor app reads it and `POSTs` daily totals.
- **iOS:** Screen Time (`DeviceActivity/FamilyControls`) is heavily restricted — data largely cannot leave the device.
- **Server-side shortcut:** **RescueTime** offers a cross-platform cloud API for productivity/active time — a way to ship screen-time-style goals *before* building the native usage-stats path.

5.3 Catalog of useful data platforms

Server-side (cloud API from the VPS — ship these first):

Platform	Goal type it powers	Auth
GitHub / GitLab	Commits, PRs, project streaks	Username / OAuth
WakaTime	Coding time per language / project	OAuth
Strava	Runs, rides, distance (no native needed)	OAuth
Fitbit / Oura / Withings	Steps, sleep, heart rate (cloud route)	OAuth
Spotify	Listening, new-artist discovery	OAuth
Goodreads / Hardcover / OpenLibrary	Books read, reading pace	Key / username
LeetCode	Problems solved, contest streaks	Username (unofficial)
RescueTime	Productive vs distracting time	API key
Todoist / Notion / Google Calendar	Task & schedule sync	OAuth
Plaid / bank APIs	Budgeting, savings targets	OAuth
OpenWeather	Ambient context (enriches goals)	API key

Device-side (require the native layer):

Source	Goal type it powers	Platform
Health Connect	Steps, sleep, heart rate, workouts	Android
UsageStatsManager / Digital Wellbeing	Screen time, app usage	Android
HealthKit	Health & fitness metrics	iOS
Location / geofencing	Place-based habits, check-ins	Android / iOS

Note on health sources: Strava, Fitbit, Oura, and Withings expose *cloud* APIs, so activity/health goals can be shipped **server-side via OAuth** — a faster route than the native Health Connect path, which remains the option for Samsung Health specifically.

5.4 Designing for “add more later”

To keep onboarding new sources cheap, treat every integration as a uniform *provider*:

- A **providers** table (name, kind: **server/device**, auth type) and per-user **connections** (tokens / usernames).
- A normalized **POST /api/metrics** ingestion shape so both scheduled server-side pulls and device-side pushes write the same way.
- Goals reference a **metric** (e.g. **github.commits**, **screen.minutes**, **steps**) rather than a hard-coded source — so a new platform is mostly config, not new plumbing.

6 Recommended Sequencing

1. **VPS + HTTPS + reverse proxy** (Caddy serving `dist/`, proxying `/api`).
2. **Tier-1 PWA** (installable, push notifications) — falls out of step 1.
3. **Server-side integrations** (GitHub first, then WakaTime / Strava / RescueTime) — highest value per effort, no native code.
4. **In-app AI assistant** via Claude API function-calling over existing endpoints.
5. **Capacitor wrap + Health Connect** — first real native step; unlocks activity goals.
6. **Device-side screen time** (UsageStatsManager) once the native layer exists.
7. **Wear OS companion** — last, narrowest scope.

6.1 Cheap backend decisions to make now

- **Model goals with a tracking source** — e.g. `manual` vs `activity_metric` (steps, distance, active minutes), so activity goals can later auto-update.
- **Keep an ingestion endpoint in mind** — e.g. `POST /api/activity` accepting metric samples; the native layer feeds it later.

The three services and the backend-as-hub model stay constant. Everything new — PWA, VPS, native app, watch, assistant, and every data integration — attaches around that core as a uniform provider rather than replacing it. Each source added makes the combined whole more unique.